

WorldSpot

Географическая игра в духе GeoGuessr

Python + rpgame · панорамы · карта · статистика · рейтинг

Методическое пособие v1.0

Содержание

1. Введение: что такое WorldSpot
2. Анатомия игры в стиле GeoGuessr
3. Подготовка окружения
4. Структура проекта WorldSpot
5. База локаций: data/locations.json
6. Игровой цикл и сцены
7. Панорама: отображение локации
8. Карта мира и выбор точки
9. Система очков: формула Haversine
10. Экран результатов раунда
11. Статистика по странам и регионам
12. Режимы: обычный и рейтинговый
13. История игр и анализ
14. Достижения и рекорды
15. Минималистичный интерфейс
16. Полный цикл раунда в коде
17. Продвинутый уровень: API панорам
18. Git и публикация на GitHub
19. Дорожная карта развития
20. Приложение: управление в шаблоне
21. Заключение

Введение: что такое WorldSpot

WorldSpot — географическая игра, вдохновлённая GeoGuessr. Игрок видит панораму улицы или природы и должен указать на карте мира, где, по его мнению, сделан снимок. Чем ближе угадывание к реальной точке, тем больше очков.

Это пособие проведёт вас от пустой папки до работающего прототипа на Python и pygame и опубликованного репозитория на GitHub. Мы создаём не клон GeoGuessr, а свою игру с уникальной статистикой: процент успеха по странам и регионам вместо флагов.

Особенности WorldSpot

- Знакомый геймплей: панорама → догадка на карте → результат.
- Минималистичный современный интерфейс.
- Обычный и рейтинговый режимы.
- Подробная статистика после каждого раунда.
- Процент угадываний по странам и регионам (без флагов).
- История игр, достижения и личные рекорды.

Что вы создадите

- Окно игры с панорамой и интерактивной картой мира.
- Систему очков по расстоянию (формула Haversine).
- Экран результатов с точностью в км и очками.
- Таблицу статистики: % по странам, лучшие и худшие регионы.
- Сохранение истории и достижений в JSON.

Практика: Старт

1. Установите Python 3.10+ и создайте папку WorldSpot.
2. Откройте проект в VS Code или Cursor.
3. Прочитайте README.md в корне шаблона.

Анатомия игры в стиле GeoGuessr

Классический раунд GeoGuessr состоит из пяти этапов: показ локации, исследование (поворот камеры, движение — в полной версии), выбор точки на карте, подсчёт очков, показ правильного ответа. WorldSpot реализует ядро этого цикла.

Игровой цикл раунда

1. PANORAMA — игрок изучает изображение локации.
2. GUESS — клик по карте мира, подтверждение.

3. REVEAL — линия от догадки до правды, расстояние, очки.
4. STATS — обновление % по стране и региону.
5. NEXT — следующая локация или итоги серии.

Система	Назначение	Сложность
Game Loop	Сцены и переключение режимов	Низкая
Locations DB	JSON с lat/lon, страной, панорамой	Низкая
Map Widget	Клик → координаты	Средняя
Scoring	Haversine + очки	Средняя
Stats Engine	% по странам/регионам	Средняя
History + Achievements	Прогресс игрока	Низкая

Совет: Начните с 5–10 тестовых локаций в JSON. Панорамы — любые JPG 1280×720 с указанием координат в метаданных.

Подготовка окружения

```
cd WorldSpot
python3 -m venv .venv
source .venv/bin/activate
pip install pygame pillow
pip freeze > requirements.txt
```

```
python3 -c "import pygame; from PIL import Image; print('OK')"
```

Важно: Для реальных уличных панорам позже подключите Mapillary API или Google Street View Static API — нужны ключи и соблюдение лицензий.

Структура проекта WorldSpot

```
WorldSpot/
|-- assets/
|   |-- panoramas/      # JPG панорамы локаций
|   +-- ui/              # карта мира, иконки
|-- data/
|   |-- locations.json # база точек
|   |-- achievements.json
|   +-- config.json    # режимы, лимиты
|-- src/
|   |-- main.py
|   |-- settings.py
|   |-- game.py
|   |-- scenes/        # menu, round, results, stats
|   +-- systems/       # scoring, stats, save, locations
|-- docs/              # эта методичка
|-- .gitignore
|-- requirements.txt
+-- README.md
```

```
# src/settings.py
WIDTH, HEIGHT = 1280, 720
FPS = 60
TITLE = 'WorldSpot'
MAX_SCORE = 5000
MAP_RECT = (40, 400, 500, 260) # x,y,w,h
PANORAMA_RECT = (40, 40, 800, 340)
ROUND_TIME_RANKED = 180 # секунд
```

База локаций: data/locations.json

Каждая локация — запись с координатами, страной, регионом и файлом панорамы. Именно здесь хранится «правда», с которой сравнивается догадка игрока.

```
{
  "locations": [
    {
      "id": "oslo_01",
      "lat": 59.9139,
      "lon": 10.7522,
      "country": "Норвегия",
      "region": "Скандинавия",
      "panorama": "oslo_fjord.jpg",
      "difficulty": "medium"
    },
    {
      "id": "tokyo_01",
      "lat": 35.6762,
      "lon": 139.6503,
      "country": "Япония",
      "region": "Восточная Азия",
      "panorama": "tokyo_street.jpg"
    }
  ]
}
```

```
# src/systems/locations.py
import json
from pathlib import Path
from random import shuffle

def load_locations():
    path = Path('data/locations.json')
    return json.loads(path.read_text(encoding='utf-8'))['locations']

def pick_round(locations, count=5):
    pool = locations.copy()
    shuffle(pool)
    return pool[:count]
```

Игровой цикл и сцены

```
# src/main.py
import pygame
from game import Game
from settings import WIDTH, HEIGHT, FPS, TITLE

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption(TITLE)
    clock = pygame.time.Clock()
    game = Game(screen)
    running = True
    while running:
        dt = clock.tick(FPS) / 1000.0
        for e in pygame.event.get():
            if e.type == pygame.QUIT: running = False
            game.handle_event(e)
        game.update(dt)
        game.draw()
        pygame.display.flip()
    pygame.quit()
```

Game переключает сцены: MenuScene → RoundScene → ResultsScene → StatsScene. Каждая сцена знает, как рисовать UI и обрабатывать клики.

Панорама: отображение локации

В прототипе панорама — изображение JPG, загруженное через `pygame.image.load`. Масштабируйте его под `PANORAMA_RECT`, сохраняя пропорции (letterbox).

```
def load_panorama(filename):
    path = ASSETS / 'panoramas' / filename
    img = pygame.image.load(str(path)).convert()
    return pygame.transform.smoothscale(img, (w, h))

def draw_panorama(surface, image, rect):
    surface.blit(image, rect.topleft)
    pygame.draw.rect(surface, (255, 255, 255), rect, 2)
```

Совет: Добавьте затемнённые полосы по краям и подпись «Где это?» — минималистичный UI сразу выглядит современнее.

Карта мира и выбор точки

Карта — прямоугольное изображение мира (экваториальная проекция). Клик (mx, my) внутри `MAP_RECT` переводится в широту и долготу.

```
def pixel_to_latlon(mx, my, map_rect):
    x = (mx - map_rect.x) / map_rect.w
    y = (my - map_rect.y) / map_rect.h
    lon = x * 360.0 - 180.0
    lat = 90.0 - y * 180.0
    return lat, lon

def latlon_to_pixel(lat, lon, map_rect):
    x = (lon + 180.0) / 360.0
    y = (90.0 - lat) / 180.0
    px = map_rect.x + x * map_rect.w
    py = map_rect.y + y * map_rect.h
    return int(px), int(py)
```

После клика покажите маркер (красный круг). Кнопка «Угадать» фиксирует выбор.

Практика: Карта

1. Нарисуйте карту мира 500×250 или используйте `assets/ui/world_map.png`.
2. По клику ставьте маркер и выводите lat/lon в углу экрана.
3. Заблокируйте повторный клик после подтверждения.

Система очков: формула Haversine

GeoGuessr начисляет до 5000 очков. Чем меньше расстояние между догадкой и правдой, тем выше счёт. Расстояние считают по сфере Земли — формула Haversine.

```
import math

def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    p = math.pi / 180
    a = (math.sin((lat2-lat1)*p/2)**2 +
         math.cos(lat1*p)*math.cos(lat2*p)*math.sin((lon2-lon1)*p/2)**2)
    return 2 * R * math.asin(math.sqrt(a))

def score_from_distance(km, max_score=5000):
    if km <= 0.05: return max_score
    return max(0, int(max_score * math.exp(-km / 2000)))
```

Подберите делитель (2000) под желаемую сложность: меньше — щедрее очки.

Экран результатов раунда

После угадывания покажите: расстояние в км, заработанные очки, правильную точку и линию между догадкой и истиной на карте.


```

class ResultsScene:
    def __init__(self, game, location, guess_latlon, distance_km, score):
        self.location = location
        self.guess = guess_latlon
        self.distance_km = distance_km
        self.score = score

    def draw(self, surface):
        draw_map_with_markers(surface, self.guess, self.truth)
        draw_text(surface, f'{self.distance_km:.0f} км · {self.score} очков')

```

Статистика по странам и регионам

Вместо флагов WorldSpot показывает процент успешных угадываний. Успех — если расстояние меньше порога (например, 500 км) или очки выше 4000.

```

# src/systems/stats.py
class RegionStats:
    def __init__(self):
        self.by_country = {} # страна -> {correct, total, avg_km}

    def record(self, country, region, distance_km, score):
        c = self.by_country.setdefault(country, {'ok':0, 'n':0, 'km':[]})
        c['n'] += 1
        c['km'].append(distance_km)
        if distance_km < 500 or score >= 4000:
            c['ok'] += 1

    def percent(self, country):
        c = self.by_country[country]
        return 0 if c['n']==0 else round(100*c['ok']/c['n'])

```

Что показывать игроку

- Процент угадываний по каждой стране.
- Средняя точность (среднее расстояние в км).
- Лучшие регионы (высокий %) и худшие (низкий %).
- График прогресса по датам (из history.json).

Страна	Игр	Успех %	Ср. км
Норвегия	12	67%	420
Япония	8	38%	2100
Бразилия	5	80%	310

Режимы: обычный и рейтинговый

Обычный режим — без таймера, можно изучать панораму сколько угодно. Рейтинговый — лимит времени на раунд (3 минуты), бонус за скорость.

```
class RoundScene:
    def __init__(self, game, location, ranked=False):
        self.ranked = ranked
        self.time_left = ROUND_TIME_RANKED if ranked else None

    def update(self, dt):
        if self.ranked and self.time_left is not None:
            self.time_left -= dt
            if self.time_left <= 0:
                self.auto_submit_worst_guess()
```

История игр и анализ

```
# savegame / history.json
{
  "games": [
    {
      "date": "2026-06-15",
      "mode": "ranked",
      "total_score": 18420,
      "rounds": [
        {"country": "Норвегия", "km": 120, "score": 4850}
      ]
    }
  ]
}
```

Экран «История» читает этот файл и рисует список прошлых сессий с датой, режимом и суммой очков. Клик — детали каждого раунда.

Достижения и рекорды

```
{
  "achievements": [
    {"id": "first_5000", "title": "Идеально!", "desc": "5000 очков в раунде"},
    {"id": "globe_trotter", "title": "Глобус", "desc": "10 разных стран"},
    {"id": "europe_master", "title": "Европа", "desc": "80% в Европе"}
  ]
}
```

После каждого раунда проверяйте условия и разблокируйте достижения. Рекорды: лучший суммарный счёт за 5 раундов, минимальное среднее расстояние.

Минималистичный интерфейс

- Фон: тёмно-серый #1a1a2e, текст белый, акцент зелёный #1a6b4a.
- Шрифт: системный sans-serif, крупные заголовки, мало декора.
- Кнопки: прямоугольники с тонкой обводкой, hover — светлее.
- Никаких лишних рамок — панорама и карта занимают 90% экрана.

```
COLORS = {  
    'bg': (26, 26, 46),  
    'accent': (26, 107, 74),  
    'text': (240, 240, 245),  
    'muted': (160, 160, 180),  
    'marker_guess': (239, 68, 68),  
    'marker_truth': (34, 197, 94),  
}
```

Полный цикл раунда в коде

1. MenuScene: выбор «Играть» / «Рейтинг» / «Статистика».
2. RoundScene: загрузка location, показ панорамы и карты.
3. Игрок кликает карту → нажимает «Угадать».
4. scoring.haversine_km + score_from_distance.
5. stats.record(country, region, km, score).
6. ResultsScene → кнопка «Далее» или итог серии.

Совет: В шаблоне WorldSpot этот цикл уже работает — запустите `python -m src.main`.

Продвинутый уровень: API панорам

Для настоящих уличных панорам используйте Mapillary API (бесплатный tier) или Google Street View Static API. Запрос по lat/lon возвращает изображение.

```
# Псевдокод Mapillary  
url = f'https://graph.mapillary.com/images?access_token=TOKEN'  
      f'&fields=id&closeto={lon},{lat}'  
# Скачать thumb URL и кэшировать в assets/panoramas/
```

Важно: Храните API-ключи в `.env`, не коммитьте в `git`.

Git и публикация на GitHub

```
git init
git add .
git commit -m 'WorldSpot: geographic guessing game prototype'
git remote add origin https://github.com/USER/WorldSpot.git
git push -u origin main
```

```
# .gitignore
.venv/
__pycache__/
.env
savegame.json
history.json
```

Дорожная карта развития

Неделя	Задача	Результат
1	Окно + панорама + карта	Клик по карте
2	Haversine + очки	Счёт раунда
3	Статистика %	Таблица стран
4	Режимы + таймер	Рейтинг
5	История + достижения	Прогресс
6	GitHub + UI полировка	Релиз

Приложение: управление в шаблоне

Действие	Управление
Клик по карте	ЛКМ
Подтвердить угадывание	Enter / Пробел
Меню	Esc
Следующий раунд	N

- Документация pygame: <https://www.pygame.org/docs/>
- GeoGuessr для вдохновения: <https://www.geoguessr.com/>
- Mapillary API: <https://www.mapillary.com/developer>
- OpenStreetMap: <https://www.openstreetmap.org/>

Заключение

WorldSpot — это исследование мира через код. Вы научились строить панорамы, карту, географический скоринг и аналитику по странам. Развивайте географическую интуицию в игре и делитесь проектом с друзьями на GitHub.

Запустите шаблон, пройдите 5 раундов, откройте статистику — и добавьте свою первую локацию в `data/locations.json`. Удачи в разработке!